

Coordinating Multi-Vendor AI Agent Fleets

Design-science findings from a home-grown probe into multi-vendor agent coordination — working draft

rendered 2026-07-08

Contents

Design-science findings from building and operating a home-grown coordination substrate as a probe into multi-vendor agent coordination (the system is the instrument, not the subject)	2
Abstract	2
1. The shift: code is getting cheap, coordination is getting expensive	3
The coordination stack	4
2. Method and evidence grades	4
3. L1 — The intent substrate (P2)	5
4. L2 — Substrate as governance (P1, P5)	7
5. L3 — Conflict-light collaborative state (P4)	10
6. L4 — Autonomy and role orchestration (P3, P6)	12
7. L5 — Cross-vendor fleet coordination (P8)	12
8. Orthogonal: the surface (P7)	15
9. Strongest objections and restatements	16
P3 — substrate-bounded autonomy	16
P8 — incentive-persistent cross-vendor gap	16
10. Threats to validity	17
11. Faulty assumptions this probe falsified	17
12. The cost/benefit of roll-your-own (verdict: open)	23
13. What a real study would measure	25
14. Related work	27
References	28
15. What the probe produced — distinctive value, honestly assessed	28
Appendix A — Proposition ↔ layer ↔ mechanism ↔ grade	32

Design-science findings from building and operating a home-grown coordination substrate as a probe into multi-vendor agent coordination (the system is the instrument, not the subject)

Status: Draft basis for a research paper · **Date:** 2026-07-08 (inception 2026-06-16) · **Genre:** Design-science / autoethnographic systems research · **Spine:** Multi-vendor fleet coordination (apex of a layered coordination stack) · **Frame:** AIDA as research probe — the deliverable is knowledge about the problem plus a roll-your-own cost/benefit, *not* a case for AIDA

This is a working draft meant to be handed to collaborators and revised. Claims are tagged with an **evidence grade** (see §2) so reviewers can see exactly how much weight each carries. Nothing here is asserted as a controlled empirical result; the contribution is the artifact and the design principles extracted from building it.

Abstract

As large language models commoditize code generation, the binding constraint on software production moves *off* the model and *onto* the coordination of many agents — across roles, across time, and, increasingly, across **vendors**. We report on AIDA, a system built to make a fleet of confident, forgetful, concurrent, multi-vendor coding agents collaborate against a shared, durable model of intent. AIDA is git-canonical: every requirement is a YAML object on an orphan branch, related agents read and write it through a typed graph and an MCP server, and code links back to it through inline trace comments. We frame AIDA’s machinery as a **coordination stack** and advance the position that, as lower layers of this stack are absorbed by model vendors, durable value migrates upward — and that the topmost layer, *cross-vendor fleet coordination*, is structurally under-provided for incentive rather than capability reasons. **AIDA is the instrument, not the subject:** we use a home-grown system as a probe to surface the real requirements, minefields, and faulty assumptions that appear only once you build, and to weigh the cost/benefit of roll-your-own — for which our honest verdict is *open*. We extract eight design propositions about the problem space (not arguments for AIDA), each embodied in a running mechanism and triangulated against market observation. A first round of headless cross-vendor experiments (a Claude-vs-Codex bake-off, deterministically graded; a five-cell gate-vs-rule ablation; a roll-your-own friction benchmark pitting the typed graph against mark-down+git+grep at scale; a cross-vendor portability pilot timing a new vendor’s zero-integration onboarding; and a 72-cell agent-surface benchmark pricing the typed/MCP surface against the plain CLI) has begun to move several of these claims from argument toward evidence, and the through-line they sharpen is this: **the shared substrate, not the brief and not the model, is the lever**. Two vendors handed the same open brief converge on the same design because the substrate dictates the shape — so owning the substrate shapes the fleet’s output regardless of vendor, and multi-vendor competition pays as a quality-variance QA mechanism rather than as design diversity. The same experiments scope our sharpest governance claim honestly — and the way they did it is the probe working as intended: we pre-registered *five* single-variable conjectures in sequence and falsified all five. Rules-just-fail fell to the I1 pilot: a capable model honored an immediate format rule at the ceiling, the gate idle. Attention-distance (a gate beats a stated rule only when the invariant sits far from the point of action) fell to I2: a buried, semantic, never-restated invariant held at the ceiling, exactly where it was predicted to leak. Vendor (the held/dropped split tracks which model executes) fell to cross-vendor I2: the same trial re-run with Codex *too* held at the ceiling (rule-only 100%, gate-saves 0%, identical to Claude). Invariant-type (output-shape rules self-comply, procedural/tool-use rules are where the gate earns its place) fell to I3: a *controlled* procedural invariant — “run an extra external step before committing,” deterministically graded on whether the step was performed — *also* held at the ceiling (rule-only 100% (10/10), gate-saves 0%). Finally **task-complexity / cognitive-load** — the leading remaining candidate, and the one we built the decisive test for — fell to

I4: the same procedural invariant and same deterministic grader as I3, but a *complex* multi-file/multi-function task with the rule buried as one of ~8 ambient instructions, never restated, held at the ceiling too (Claude, n=10/arm, Arm-R 100% (10/10), gate-saves 0). There are now **five controlled cells (I1, I2, I2-codex, I3, I4) and every one is at the ceiling — rule honored, gate idle**, across two vendors and from trivial to complex; the only rule-drop in the whole program is a single, uncontrolled bake-off data point. Every single-variable conjecture is now spent, so the conclusion is **methodological and definitive, not a sixth theory: a clean ablation cannot reproduce rule-dropping at all**. Five increasingly adversarial designs produced no signal. The one drop lives in a condition controlled designs structurally cannot reach — a *large, pre-existing, real codebase* under long-horizon work, where attention is contended by the existing system’s complexity, not a synthetic brief; I4 scaled the synthetic task as far as a deterministically-graded ablation can (multi-file, multi-function, buried rule) and it *still* held, so the residual cause, if real and not n=1 noise, is a property of the messy real environment, not of any task you can write down. The honest pivot is therefore **not a sixth ablation** (it would hit the same ceiling) but **field instrumentation** — counting stated-rule violations in real aida queue work --auto-complete drains on real repos and correlating with repo size / task span / context pressure: a different *kind* of evidence. That instrumentation has since shipped its first two slices (SPIKE-67): the git log itself is the sensor, and on this repo’s own 750-commit history the apparent rules-fail-under-load signal *does not survive the controls* — a squash-merge artifact accounted for ~3 of every 5 apparent misses, holding commit type fixed flattens the span curve, and orchestrator-drained commits adhere *better* (2% would-block) than interactive ones (17%). Lab and field now agree on a null for this repo; the open path narrows to a cross-repo harvest over larger, less-disciplined histories. The only P1 claim with evidence is the strong **negative** — for everything controllably measurable (five cells, two vendors, trivial→complex), a capable 2026 model honors a stated rule at the ceiling and the gate buys nothing; the original “a gate is *required* to hold an invariant against a capable LLM” is falsified across every controlled condition, surviving only in the unmeasured real-repo/long-autonomy regime, against which AIDA’s own hard gates should now be audited. Two further pilots tilt the broader verdict from open toward open-but-better-instrumented: a roll-your-own friction benchmark (same specs at 10/50/200/500, deterministic grading) shows the typed graph *pays* on relational queries (~80× latency at 500) and rename blast-radius (silent reference-rot at every scale ≥ 50 that the id-stable graph prevents) while grep *wins* on full-text and flat enumeration — so the “the rich layer is dead weight” reading is falsified for relational/rename work and conceded for textual work, on capability-friction with the operator-discipline confound named not escaped; and a cross-vendor portability pilot shows a brand-new vendor (Codex) joining a live fleet with **zero bespoke integration**, riding discover→claim→ship→verify in ~50 s on the stock CLI alone — the onboarding-cost premise single-vendor incumbents structurally cannot demonstrate — at n=1 vendor on one trivial task. A separate operating-cost round prices the fleet itself and finds the costs measurable and movable: the typed/MCP agent surface costs ~2× the plain CLI for equal-or-worse success on identical tasks (structural — on-demand schema loading does not rescue it), recycled warm worktrees cut per-agent build cost ~30×, and event-driven supervision replaces timer polling to cut watch cost from O(time) to O(real events) — so the practical agent surface for fleet work is the token-efficient CLI, and the typed protocol is an option you pay a premium for, not a default. We are explicit about the threats to validity of single-system, single-operator, alpha-stage design research, and we state what a multi-team study would have to measure to confirm or refute each proposition.

1. The shift: code is getting cheap, coordination is getting expensive

The first-order effect of capable coding agents is obvious — code gets written faster. The second-order effect is the interesting one: when any unit of code is cheap to (re)generate,

the *code* stops being the scarce, durable artifact. What becomes scarce is an **addressable, queryable model of intent** that survives the churn — what was decided, why, what depends on what, which code satisfies which requirement, and what is still open. A human team carried that model implicitly, in heads and in review. A fleet of agents cannot: agents are **confident** (they assert), **forgetful** (each session is a fresh context), and **concurrent** (many act on the same artifacts at once). And the fleet is increasingly **multi-vendor** — a Claude implementer, a Codex sibling, a Gemini reviewer — none of which shares the others’ memory, conventions, or governance.

AIDA is a home-grown attempt at the missing substrate — and we treat it as a **probe**, not a product to defend. This paper is not a tool description, nor a case for AIDA; it uses what building (and dogfooding) AIDA taught us to map *which layer of the agent-coordination problem holds durable value*, what a home-grown solution can and cannot reach, and whether rolling your own is worth the cost. The artifact is the instrument; the deliverable is the knowledge.

AIDA as probe, not product. Throughout, read every mechanism as evidence about the *problem*, not as a feature pitch. Where AIDA succeeds it marks what is possible; where it strains or fails — a scaffolding bug that polluted its own main branch mid-writing (BUG-570), weeks lost to a market-discovery blind spot — it marks a minefield or a true cost of rolling your own. Both directions are data. The right answer the probe points to may well be “do not build this,” or “the tool is not AIDA.”

The coordination stack

L5	CROSS-VENDOR FLEET COORDINATION	(mailbox, roles, leases)	← apex / the bet
L4	AUTONOMY & ROLE ORCHESTRATION	(cascade, punt-continue)	
L3	COLLABORATIVE STATE	(conflict-light git store)	
L2	ENFORCEMENT & GOVERNANCE	(substrate-as-bouncer)	
L1	INTENT SUBSTRATE	(graph, IDs, traces, MCP)	
L0	MODEL / AGENT CAPABILITY	(vendor turf, commoditizing)	

The central research question. Value in this stack is migrating upward: as model vendors solve L0 and begin to absorb L1-L2 inside their own walled gardens (project memory, skills, sub-agents, internal task graphs), the durable, defensible value moves to the layers a single vendor is *disincentivized* to provide well — the cross-vendor layers L4-L5. For a large corporation deciding where to invest as the ground shifts, the open question is **how far up the stack value has already moved, how fast, and whether building any of it yourself is worth the cost**. We do not claim certainty about the answer — and we explicitly do not claim AIDA is the thing to build. We claim the stack is the right way to *ask*; that the incentive structure makes L4-L5 the layers a single vendor will under-serve; and that whether *you* should roll your own there is a cost/benefit question we leave open (§12).

2. Method and evidence grades

This is **design-science research** (Hevner): the artifact is the primary contribution, and the knowledge claims are design principles extracted from building and operating it. It is also

autoethnographic — AIDA is built *using* AIDA; the dogfood is the experiment, and the authors are the subjects. This is a legitimate genre (systems “experience reports,” design science, reflective practice), but it has sharp limits, which we state up front rather than bury (§10).

Each proposition carries an **evidence grade**:

- **(M) Mechanistic** — argued from first principles about how LLM agents behave; the claim says *why* it must hold, independent of how often we have seen it.
- **(D) Dogfood** — observed repeatedly while building AIDA with AIDA; real but autoethnographic and uncontrolled.
- **(K) Market** — triangulated against external behavior (what competitors build, what users adopt); demand-side evidence, not internal.

A strong claim is one that holds on more than one grade. The strongest here — substrate-as-bouncer (P1), substrate-bounded autonomy (P3), conflict-light state (P4) — are **(M)+(D)**.

3. L1 — The intent substrate (P2)

P2 (Intent as the durable layer) — after red-team, split (like P8). The unscoped form — “*the model, not the code, is the thing worth version-controlling carefully*” — is **not supported** by what we have; a red-team pass (§13, TASK-846) separated the part the dogfood actually carries from the part that remains an unproven bet, and the honest paper asserts only the first:

- **P2a (holds, grade (M)): a stable, mutually-addressable name per unit of work is necessary coordination infrastructure** for a forgetful, concurrent, multi-vendor fleet. A code symbol is not a work-unit handle: it is renamed, deleted, and split, it lives inside one vendor’s session, and it cannot name a *not-yet-written* unit of work at all. A stable ID is the only thing a forgetful agent and a concurrent sibling in a *different* vendor can both name without ambiguity across sessions. This is the floor, and it is mechanistically sound.
- **P2b (now with controlled evidence for the relational/rename half, grade (D); con-found named not escaped): whether the rich layer — typed relationships, bidirectional trace comments, the MCP graph — pays for its maintenance cost over plain unstructured-markdown + git + grep was the paper’s biggest open question (§12); the RYO friction benchmark moves it from “unproven” to a controlled both/and.** A deterministically-graded benchmark (ablations/2026-06-19-ryo-friction-benchmark.md, TASK-871) ran the *same* synthetic spec set at 10/50/200/500 specs through two arms — one-markdown-file-per-spec + git + grep (the honest RYO setup) vs. the same specs in an aida git-canonical store — across five tasks, deliberately including two **fairness** tasks where grep should tie or win. The result splits cleanly by **question-shape, not by graph-vs-grep**: the typed layer **pays on relational queries** (transitive blocked-closure latency ~80× faster at 500 specs, flat vs. grep’s roughly-linear climb, both correct) and **pays decisively on rename blast-radius** (a naive RYO rename silently leaves 2 dangling cross-references at every scale ≥ 50 — no scale floor — while AIDA’s id-stable edges miss zero); but **grep genuinely wins** on full-text search and flat enumeration (faster, zero-install). So the skeptic’s “the rich layer is dead weight” reading is **falsified for relational and rename-sensitive fleet work** and **conceded for plain textual/enumerative work** — the honest posture is both/and by task shape, not “AIDA replaces grep.” The **operator-discipline confound (§10) is named, not escaped**: single-operator, synthetic specs, so the benchmark measures *capability-friction* (what the tooling makes easy or hard), not *team-adoption* (whether humans keep markdown cross-refs consistent by hand — exactly the thing the graph automates). Two earlier red-team undercuts survive intact and bound the win: (i) the open-brief convergence finding (below, §3) shows agents converging off the *codebase’s physical shape* — queues, leases, drain locks, paths — **not** off the typed graph, so that evidence still does not isolate the graph; (ii) trace comments rot **invisibly** —

a renamed, deleted, or renumbered spec leaves a `// trace:` pointing at nothing and nothing goes red (observed in this repo, including provenance-stealing renumbering), whereas a stale type fails to compile and a stale test goes red, so the separately-maintained model is *less* self-healing than the code it annotates. The benchmark cuts the other way on (ii): it shows the *cost* of that rot — at fleet scale a roll-your-own rename rots references silently while the id-stable graph cannot — so the rot is a real RYO liability, not merely an AIDA one.

Mechanism in AIDA. Every requirement is a YAML object (`objects/TYPE/000/SPEC-ID.yaml`) on the orphan `aida-store` branch, with a stable ID that never moves. Requirements carry typed relationships (`blocks`, `blocked-by`, `parent`, `depends-on`) that form a graph queryable with `aida graph <ID> --impact`. Code links back through inline `// trace:SPEC-ID` comments, making the satisfaction relation *machine-checkable from either direction*. An MCP server exposes the whole graph to any agent as tools and resources, so the model is not a document an agent must be told to read — it is an API the agent queries. (Measured caveat, 2026-06-29: on identical substrate reads the MCP surface costs $\sim 2\times$ the plain CLI for equal-or-worse success — see §8 — so in practice the typed API is the *option* and the token-efficient CLI is the default agent surface; the graph pays, but the cheapest access path to it is text.)

Why it matters for a fleet. A stable ID is the only thing a forgetful agent and a concurrent sibling can both name without ambiguity. The graph is what lets a cold-booting agent reconstruct *context* it never held. The trace comment is what lets a reviewer verify “done” against code rather than against a claim (a failure mode we hit repeatedly — agents mark work done that the diff does not contain).

What would falsify P2b — and what the benchmark already settled. The RYO friction benchmark (above) is the first controlled cut: it falsifies the “dead weight” reading for relational and rename-sensitive work and concedes textual/enumerative work to `grep` — but on *capability-friction* only, with the operator-discipline confound named not escaped. What it does **not** settle is **team-adoption**: a multi-team study finding that fleets coordinate just as well over plain markdown + git + `grep` as over the typed graph — once humans, not a single disciplined operator, maintain the cross-references — would still kill the rich layer’s case where it matters. A cheaper, internal falsifier remains: a **trace-read-rate audit** — instrument `query_graph / show_requirement` read volume against trace-write volume; if the graph is *written but not consulted* (high write, near-zero read), the typing is dead weight maintained out of discipline, not leverage. P2a (the stable name) is not at risk from any of these — it is the floor; P2b (the rich layer over markdown) is what the benchmark partially settled and the team study would finish.

A stronger reading the experiments surfaced — the substrate shapes output, not just cost. The standing P2 claim is that the substrate *reduces coordination cost*. The open-brief bake-off (2026-06-18-open-brief-convergence.md) points at something deeper. We handed Claude and Codex a deliberately **open** brief — goal stated, but command name, signals, shape, and output all left to the implementer — expecting architectural divergence. They converged: the same command name, the same axes, the same module, the same verdict model, the same signal set. Convergence was *total* exactly where the substrate dictated the answer (both read the same physical paths — queues, leases, drain locks, findings — so there was one signal set and one way to read each), and divergence survived *only* in the residual the substrate left unspecified (the rollup math, the reuse-vs-reimplement call). The corollary is testable: to induce genuine divergence you must vary the *substrate* or pose the problem before the substrate exists — loosening the brief does not. **So the substrate does not merely lower coordination cost; it determines the shape of what every vendor’s agent builds. Whoever owns the shared substrate shapes the fleet’s output, regardless of which vendor executes** — the deepest statement of AIDA’s bet, and a new (D) angle that connects directly to the apex claim (§7). **Note the scope carefully:** the shape the agents converged off is the codebase’s *physical* substrate (queues, leases, drain locks, paths), so this strengthens the L5/P8a “own the substrate” bet but does **not** isolate AIDA’s *typed graph* as the cause — which is exactly why it is folded above as an undercut to P2b, not as support for it. (Pilot, $n=1$ task, self-evaluating

judge; the brief pre-named two axes, which plausibly anchored axis-convergence but not the name/rollup/path convergence.)

4. L2 — Substrate as governance (P1, P5)

P1 (Substrate-as-bouncer) — grade (M)(D). The sharpest claim in the paper — and the one a pilot forced us to scope. *As originally stated:* natural-language instructions do not reliably constrain a capable LLM agent — a rule in a prompt, a CLAUDE.md, or a memory is advisory; the agent is free, confident, and will route around it when its in-context reasoning disagrees, so to **guarantee** an invariant you must place a **programmatic gate** in the execution substrate, a bouncer that refuses the action rather than a sign that asks politely. We then *ran the ablation* (below), and the blanket form did not survive — which produced a sharper claim, not a weaker paper.

Mechanism in AIDA. Status transitions are gated, not documented: approved/planned are advisor-only and the CLI *refuses* the transition from a non-advisor identity; completed is merge-driven and cannot be hand-set; a (SPEC-ID) commit trailer is what auto-completes a spec, so “done” is a property of git ancestry, not an assertion. The merge-gate assigns short IDs deterministically rather than trusting an agent to pick a free one. Each of these started life as a *rule* an agent ignored, and only became reliable once it became a gate.

After the gate-vs-rule pilot — scoped, and sharper. We ran the §13 ablation rather than asserting P1 from anecdote: same invariant (the commit-message format), same model, same task, varying *only* whether a programmatic gate (AIDA_COMMIT_STRICT) rejects violations (2026-06-18-gate-vs-rule-pilot.md; n=10/arm, deterministic grading, pre-registered interpretation). The result went *against* the blanket claim: a capable 2026 model honored the explicit in-context rule **100% (10/10)**, and the gate did **no work** — it never rejected a commit. By the pre-registered threshold (Arm-R $\geq 95\%$ $\rightarrow$ P1 weakened), **P1 is falsified for simple, immediate, in-context rules**. The 2024-vintage “agents route around rules” did not reproduce on the easy case.

The pilot tested the easiest possible case — the rule is short, explicit, and the commit is the task, so the invariant sits right at the moment of action. To explain why the original P1 evidence (and the competitive bake-off, §7, where Codex *skipped* a named rule buried as one line of fine print in a 19-line brief) showed rules leaking, we advanced a conjecture: that the lever is **attention-distance** — a capable model honors immediate, point-of-action invariants and drops buried, deferred ones, and *that* is where a gate earns its place. We pre-registered it and built the one experiment designed to overturn it.

The I2 ablation falsified the attention-distance conjecture (2026-06-18-gate-vs-rule-i2.md). I2 was engineered as the *opposite* of the pilot: a semantic, buried, never-task-restated invariant — “every code change carries a // trace: comment”, an ambient CLAUDE.md rule applied deep inside a code task — i.e. deliberately *high* attention-distance. The conjecture predicted the rule would start to leak and the gate would log saves. It did not. **n=10/arm, 20 live headless claude -p runs (2026-06-18): Arm-R (rule-only) landed-compliance 100% (10/10); Arm-G gate-saves 0% (0/10).** Even the buried semantic rule held at the ceiling, and the gate did *exactly zero* work — identical to the immediate case. By the pre-registered threshold (Arm-R $\geq 95\%$ $\rightarrow$ conjecture weakened), the attention-distance conjecture does not survive. This is a clean negative, not a partial confirmation: the variable that *was* supposed to separate held-from-dropped invariants does not.

The cross-vendor I2 ablation then falsified the vendor conjecture too (2026-06-18-gate-vs-rule-i2.md). We ran I2 unchanged with Codex (codex exec) as the implementer — the one experiment the vendor hypothesis predicted would leak (Codex’s rule-only adherence drops below Claude’s, the gate logs saves). It did not: **n=10/arm, Arm-R (rule-only) 100% (10/10),**

Arm-G gate-saves 0% (0/10) — identical to Claude. Both vendors honored the buried ambient trace rule at the ceiling; the gate did zero work in all three cells run so far (the immediate I1 pilot, I2-Claude, I2-Codex). So vendor does not separate held-from-dropped either; *two* pre-registered conjectures — attention-distance and vendor — were now both cleanly falsified, and two more (invariant-type, then task-complexity) were about to fall the same way.

I3 falsified the invariant-type conjecture; I4 then falsified task-complexity — the last candidate (2026-06-18-gate-vs-rule-i3.md, 2026-06-18-gate-vs-rule-i4.md). Invariant-type (output-shape rules self-comply, procedural/tool-use rules are where the gate earns its place — the explanation the I1/I2 results pointed at) fell to **I3**: a *controlled* procedural invariant — “before committing, run the provided `./record-change.sh <fn> script,`” graded deterministically on whether the step was performed, not on the shape of the code — held at the ceiling exactly like the output-shape ones (**n=10/arm: Arm-R rule-only 100% (10/10), Arm-G gate-saves 0% (0/10)**). That left one dimension separating every controlled cell from the lone drop: the cells were all **trivial single-function tasks**; the drop was a **complex, multi-step, real-codebase task**. So we built the decisive test. **I4** reuses I3’s procedural invariant *and* its deterministic grader unchanged, varying *only* task complexity: each trial implements a small but multi-step, multi-file module (several functions across `src/lib.rs` + `src/<module>.rs`) behind a long ~20-line brief with ~8 competing instructions, the record rule buried among them and never restated — maximizing attention-distance and cognitive load at once. The complexity/cognitive-load hypothesis predicted the buried rule would finally leak and the gate would log saves. It did not: **Claude, n=10/arm, Arm-R (rule-only) 100% (10/10), Arm-G gate-saves 0% (0/10)**. All ten complex trials ran `./record-change.sh` and landed the entry; the gate fired zero rejections. By the pre-registered threshold (Arm-R $\geq 95\%$ → hypothesis weakened), task-complexity does not survive either.

Five controlled cells now bracket the question, and the lesson they teach is *methodological and definitive* — every one of them sits at the ceiling, so the design that produced them cannot tell us what makes a rule drop at all:

Cell	Invariant	Distance	Vendor	Invariant type	Complexity	Rule-only adherence
I1 pilot	commit-message format	low	Claude	output-shape	trivial	100%
I2	trace-coverage on code	high	Claude	output-shape	trivial	100%
I2 cross-vendor	trace-coverage on code	high	Codex	output-shape	trivial	100%
I3	run <code>record-change.sh</code> before commit	high	Claude	procedural/tool use	trivial	100%
I4	run <code>record-change.sh</code> before commit	high	Claude	procedural/tool use	complex	100%
Bake-off (uncontrolled)	use the <code>--ai</code> gate (fine print)	high	Codex	procedural/tool use	complex, real repo	dropped (n=1)

Five single-variable conjectures were pre-registered and falsified in sequence, each drawn through that one bake-off drop: *rules-just-fail* (I1), *attention-distance* (I2), *vendor* (cross-vendor I2), *invariant-type* (I3), and *task-complexity* (I4). In not one controlled condition did a capable 2026 model drop a stated rule; in not one did the gate do any measured work. I4 scaled the synthetic task as far as a clean, deterministically-graded ablation can — multi-file, multi-function, buried rule — and it *still* held, so the cells do not differ from the lone drop on any dimension you can write into a task brief.

The honest reading is therefore no longer a candidate-narrowing exercise but a **definitive methodological verdict: a clean ablation cannot reproduce rule-dropping at all**. Five increasingly adversarial designs — varying the invariant’s type, its distance from the action, the vendor, and the task’s cognitive load — each hit the ceiling, no signal. The one drop in the entire program lives in a condition the controlled designs structurally cannot reach: a **large, pre-existing, real codebase** under genuine long-horizon work, where the agent’s attention is contended by the *existing system’s* complexity, not a synthetic brief’s. The residual cause, if it is real and not $n=1$ noise, is **not a property of the task you can write down; it is a property of the messy real environment the task runs in** — and a controlled ablation that added that would stop being controlled. The five cells are real evidence — they replicate a genuine finding — but a finding about the floor, not about the gate.

Restated P1 — grade (M)(D), landed on the program’s terminus (five cells, one ceiling: I1, I2, cross-vendor I2, I3, I4). *The only part of P1 the evidence supports is a strong negative: for everything controllably measurable — five cells, two vendors, output-shape and procedural, near and far from the action, trivial through complex — a capable 2026 model honors a stated rule at the ceiling and the gate does zero measured work. The original blanket claim — “a programmatic gate is required to hold an invariant against a capable LLM” — is falsified across every controlled condition we could construct. The meta-finding is methodological and now firm: a clean ablation cannot reproduce rule-dropping at all, so “when does a gate beat a rule” is not answerable by synthetic ablation — we proved it constructively, five designs and no signal.*** The single rule-drop in the whole program (the bake-off, $n=1$) lives in the one place a controlled design cannot follow: a large, pre-existing, real codebase under long unattended autonomy, where attention is contended by the existing system rather than a synthetic brief. **The honest pivot is therefore not a sixth ablation — it would hit the same ceiling — but field instrumentation:** count stated-rule violations in real aida queue work --auto-complete drains on real repos and correlate with repo size / task span / context pressure. That is a different *kind* of evidence than this program could produce — and its first two slices have now run (SPIKE-67: `ablations/2026-06-20-field-instrumentation-spike-67.md` + `2026-06-22-field-study-slice2-spike-67.md`). The design treats the git log itself as the sensor (no live hooks in keystone machinery), and the slice-2 controls *reversed the slice-1 headline*: a squash-merge suffix accounted for ~3 of every 5 apparent `commit_format` misses, holding commit type to feat/fix flattens the span curve (the “load effect” was type composition), and **orchestrator-drained commits adhere better (2% would-block) than interactive ones (17%)** — the disciplined harness improves adherence rather than degrading under context pressure. First-repo verdict: **lab and field agree on a null**; a clean prose-rule-dropping signal is not reproducible here. Honest bounds: one repo, one disciplined operator — the cross-repo harvest over less-disciplined histories is what remains. **Product tie-in:** the gate’s justification survives **only** in the real-repo/long-autonomy regime — now measured *null on this repo’s own history*, still open on undisciplined external repos, so AIDA’s existing hard gates should be **audited against that bar** — the drain rails, don’t-merge-over-RequestChanges, push-and-PR atomicity (BUG-269) plausibly fire there, but any hard gate that only ever fires in small, well-scoped tasks is a removal candidate (surface bloat and false confidence with no demonstrated benefit). The clear product rule: do **not** gate output-shape or

procedural rules that fire in well-scoped work; rules suffice there.

P5 (Calibration over accuracy) — after red-team (TASK-846), scoped: record + compare are built and useful; the self-correcting-beats-static *strong form* is conjecture, not finding. The loop has three legs — **record** a prediction now, **compare** against ground truth on landing, **tune** the next estimate — and only the first two exist in the artifact. `complexity_calibration.rs` / `effort_calibration.rs` / `calibration.rs` record predictions; mismatches, deltas, and stats are surfaced to a human via `aida findings calibration --stats`. **No code closes the loop** — every read-back path terminates in a report an operator reads; nothing adjusts a *future* estimate from prior records. So AIDA ships a calibration **dashboard**, not a self-correcting **estimator**.

- **P5a (holds, grade (M)(D)): record + compare are built and genuinely useful** — making the predicted-vs-actual gap visible is real value, and we grant it freely.
- **P5b (conjecture, reduced grade): “a 60%-accurate learner beats a static 80%” is illustrative, never measured.** There is no learning estimator, no convergence curve, and no head-to-head — so the figure is a *thought experiment*, not a result. Two further caveats even if the tune leg were built: (i) **ground truth here is subjective** — a reviewer’s complexity verdict or a punt count — so the loop would converge to inter-rater *agreement*, not correctness; (ii) the **estimand is non-stationary** — the model and the codebase shift *under* the estimator — and we have no convergence-or-stationarity argument that a learner tracks a moving target. The strong form is therefore conjecture; the §10 self-evaluation-bias caveat bites here too (the ground truth is the authors’ own verdicts). The falsifiable experiment it needs is an actual **two-arm learner-vs-static ablation with the base model held fixed** (§13).

Mechanism in AIDA. Calibration mode emits two verdicts per autonomous decision (a cold-boot driver and a fork-from-live shadow), **recorded** as findings and **compared** via `aida findings calibration --stats`, deliberately to mine the gap between what the substrate predicted and what happened — but the gap is surfaced for a human to act on; nothing in the substrate consumes it to **tune** the next prediction.

Why these are the governance layer. Together they say: you do not *configure* an agent fleet’s behavior, you *fence* the parts that need fencing and then *measure* it — and the pilot adds the crucial qualifier that “the parts that need fencing” is a *triage*, not “everything.” This is the layer most directly threatened by vendors (each is building its own in-house gates and memory) — but only *within* its own walls, which is exactly the seam the upper layers exploit.

5. L3 — Conflict-light collaborative state (P4)

P4 (Conflict-light state in plain git) — after red-team (TASK-846), scoped: the append-union half holds; the HLC-determinism half is rebutted as stated. The defensible, verified claim is that **append-only event streams merged by union** keep multi-agent state conflict-*light* without a CRDT library or a database. The original second half — that scalar registers merge LWW under a (HLC-timestamp, node-id) total order, “the HLC making LWW deterministic” — does **not** describe the artifact, and the red-team verified this directly against code (BUG-578). We split the proposition:

- **P4a (holds, grade (M)(D)): append-union / G-Set is sound.** `history:`, `comments:`, and `processing_record:` merge as **union by entry UUID** (grow-only sets — commutative, associative, idempotent — so two replicas that observed the same updates compute the same value regardless of order or duplication). This is the real, defensible contribution: it is what makes git textual conflicts on these files rare, and it needs no clock and no coordinator.

- **P4b (rebutted as stated, grade (D)): scalar merge is *not* HLC-deterministic.** In the artifact the HLC is **built but unwired** — `hlc.rs` has **zero non-test call sites** (only a `pub use re-export` at `lib.rs:148`), and Requirement carries **no HLC field**. The real scalar merge (`conflict.rs:199`) is `ours.modified_at >= theirs.modified_at` — **raw wall-clock LWW, with ties resolving to “ours”** (the clone performing the merge). That is a **last-puller-wins** tie-break, not a data-function of the edits, so it is **non-deterministic across clones**: two clones merging the same pair of concurrent same-field edits in opposite roles pick opposite winners, and under clock skew the wall-clock-later edit beats the causally-later one — the exact failure the HLC was built to prevent. It silently drops the losing concurrent scalar edit *by design*. This is acceptable **today** only because the workload is partition-friendly — agents append to streams or edit different specs, so concurrent writes to the *same scalar field* are rare — not because convergence is guaranteed. (Filed as **BUG-578**; the fix is to either wire the HLC into the comparator + model + a cross-clone-skew determinism test, or remove the dead module and make the tie-break a data-function such as `node-id`.)

The conflation to avoid. “Few git textual conflicts” (true — delivered by the union) is **not** “deterministic convergence of scalars” (not delivered). The paper previously credited the HLC for the second; it earns neither, because it is not in the merge path.

Mechanism in AIDA. `conflict.rs::merge_spec_three_way` is a structured three-way merge driver for one spec’s YAML. `history:`, `comments:`, and `processing_record:` are **union by id** (G-Set semantics — order-independent, idempotent, never conflict). `tags:` is a set union. Scalar fields (`status`, `title`, `priority`, ...) are **wall-clock LWW on `modified_at`, ties → “ours”** (`conflict.rs:199`) — *not* HLC-ordered (BUG-578). The append-only *shape* is what makes the union trivial to define; the *structured driver* is what actually buys conflict-freedom for the streams — two raw appends to one file still collide under git’s line merge, so the shape alone is not enough.

The honest limitation, as a worked example of the method. The tag G-Set loses concurrent removals (a tag deleted on one node reappears after merge, because union cannot encode a delete). The principled fix is an OR-Set (observed-remove set). We filed this as an *exploratory* item rather than a mandate — illustrating the paper’s own stance that not every theoretical gap is worth closing, and that knowing *which* to leave open is itself a finding.

The formal core — and where the artifact falls short of it. For the **append-only arrays** the merge genuinely converges: each is a grow-only set (G-Set) whose join is union, with $\text{merge}(a, a) = a$, $\text{merge}(a, b) = \text{merge}(b, a)$, and $\text{merge}(\text{merge}(a, b), c) = \text{merge}(a, \text{merge}(b, c))$ holding trivially — so two replicas that have observed the same updates compute the same value regardless of order or duplication. This is the real CRDT in the system. **The scalar fields are where theory and artifact diverge.** A *proper* LWW-Register would take the value with the greater (HLC-timestamp, `node-id`) pair, and a hybrid logical clock would supply the total order that makes that join deterministic across nodes without a coordinator — but **AIDA does not implement this** (BUG-578): the actual scalar join is “take the value with the greater wall-clock `modified_at`, ties → the merging clone,” which is *not* a pure data-function (the tie-break depends on *which clone runs the merge*) and is therefore **not** a coordinator-free deterministic join. So the formal LWW-Register story is the *target*, not the shipped behavior; the shipped behavior is order-dependent under ties and skew. What git’s native three-way merge cannot do is recognize field semantics at all — it diffs lines, so two appends to one file collide — and the schema-aware driver’s real win is replacing that line-merge with the per-field **union** for the streams. **The reusable recipe, beyond AIDA:** model each field as the CRDT whose semantics match its use — event stream → G-Set (or OR-Set when deletes must survive), current value → LWW- or MV-Register — *and, for the register case, key the LWW on a logical clock (an actual HLC), not wall-clock time, or convergence is not guaranteed*. The cost is that *every* field must be classified; the unclassified default (whole-object or wall-clock LWW) silently drops concurrent edits — which is precisely the trap AIDA’s own scalar path fell into, and which the tag G-Set’s lost-removal only partially illustrates.

6. L4 — Autonomy and role orchestration (P3, P6)

P3 (Substrate-bounded autonomy) — grade (M)(D). The most counter-intuitive claim.

The quality ceiling of an autonomous decision is set by the **richness of the substrate the deciding agent boots from, not by the model’s raw capability** — because, in a fleet, each escalation/resolution step is typically a *cold boot* (a fresh process with no memory of the live session that produced the problem). Therefore the highest-leverage investment for better autonomy is *substrate enrichment*, not prompt engineering or a bigger model.

Mechanism in AIDA. When a headless implementer hits a design fork it *punts* (parks the spec NeedsAttention); the orchestrator routes the punt to a headless **advisor tier** — a fresh claude -p invocation that has only what the substrate carries, not what the live session knew. We observed directly that this advisor is a cold boot, which reframed our roadmap: the levers that matter are (1) enriching the substrate the cold-boot reads, (2) fork-from-live snapshots, (3) a persistent advisor entity — *all substrate moves*, none a model move.

After red-team — scoped restatement. The cold-boot premise is partly *self-imposed* (it follows from spawning a fresh claude -p, which fork-from-live and a persistent advisor would dissolve), and “substrate richness” and “model capability” are not independently variable — a more capable model needs less substrate to reach the same decision. So P3 is not a law of agent autonomy. Its defensible, actionable form is **scoped**: *under stateless escalation, substrate enrichment is the cheapest and most reliable autonomy lever — cheaper than chasing model upgrades you do not control*. The unscoped “substrate dominates capability” version likely holds at today’s model tier and weakens as models improve; we assert only the scoped claim. See §9.

P6 (Punt-and-continue role cascade) — grade (M)(D). Reliable autonomous throughput needs **role separation** (implementer → advisor → human) with **punt-and-continue** semantics: a blocker parks exactly one unit of work and routes it to the right role; it never halts the line. The fleet’s throughput is then bounded by the rate of genuinely-human decisions, not by the first blocker.

Mechanism in AIDA. The resilient drain (EPIC-28): a shelveable phase failure parks one spec and continues; dependents skip; the run exits with a distinct code so a supervisor can triage the parked set. The three-mode autonomy ladder separates *is a human present?* from *what should be escalated?* as orthogonal axes.

7. L5 — Cross-vendor fleet coordination (P8)

P8 (Incentive-persistent gap) — grade (M)(K). The apex claim. The durable, cross-vendor agent-coordination layer is under-provided for **incentive** reasons, not capability reasons. Each model vendor can build excellent *within-vendor* coordination (memory, sub-agents, task graphs, skills) and is strongly incentivized to make it *sticky* and *non-portable* — that is the lock-in. A coordination layer that is deliberately **vendor-neutral and durable** — a Claude agent and a Codex agent and a human sharing one mailbox, one role queue, one lease table, one intent graph — is precisely what no single vendor is incentivized to build well. The gap is therefore **structurally persistent**: it does not close as models improve, because it is not a capability gap.

Mechanism in AIDA. The store and the inter-agent surface are vendor-agnostic by construction: a file-based mailbox (send_message/read_inbox), briefs, directives, findings, leases, and roles, all exposed identically over CLI and MCP so any tool that speaks either can join the fleet.

The strategic stance we converged on: **ride the vendor's native coordination within a vendor session; own the cross-vendor, durable substrate.** Within-vendor, do not compete with the platform; across vendors and across time, be the layer the platform will not build. The stance is now load-bearing in the implementation rather than aspirational: as of July 2026 the substrate runs fleet phases on whichever vendor binaries a machine actually has — a codex-only machine gets the command set scaffolded as Codex custom prompts at init, headless phases route to an available vendor, and the doctor degrades gracefully when a binary is absent — so no single vendor's tooling is an install-time assumption.

Why the substrate is the apex lever (the convergence finding, carried up from §3). The open-brief bake-off (§3; 2026-06-18-open-brief-convergence.md) gives P8a a mechanism it lacked. Two vendors handed the same open brief converged on the same design *because the shared substrate dictated the shape* — the substrate, not the brief and not the model, is what determines the fleet's output. This is the same fact P8a turns on from the other side: the reason no single vendor builds *portable* coordination is the very reason the substrate is the lever — **the substrate is the neutral ground none of the vendors controls.** A layer that shapes every vendor's output regardless of which vendor executes is precisely the layer each vendor is disincentivized to cede to a neutral party. Own the substrate → shape the multi-vendor fleet's work; that is the strongest form of the L5 bet.

Multi-vendor competition is a QA mechanism, not a design-diversity engine. The two bake-off runs (2026-06-17-competitive-claude-vs-codex.md, 2026-06-18-open-brief-convergence.md) reframe what running N vendors is *for*. Because designs converge on a real codebase (driven by the substrate, above), the payoff of competition is **not** architectural diversity — it is **execution-quality variance within the converged design.** Run 1 (prescriptive brief) and run 2 (open brief) both produced the same design implemented with materially different correctness, and only running both arms surfaced the regression the loser shipped (run 1: Codex left a safety gate ungated and the AI path untested; run 2: Codex re-derived a liveness probe and misclassified a deleted-worktree lease, losing a state the canonical path encodes). Competition's value is therefore **selection + verification + regression-catching** — run a spec through N vendors plus a judge, ship the best, catch the one that drifted from the substrate — not emergent combination (on a prescriptive brief, the judge's synthesis was “ship the winner, optionally graft a small reuse”). A neutral substrate-owner is the only party positioned to offer this compete-style capability, and the honest product framing is quality-assurance, not design exploration. (Both runs: pilot, n=1 task each, self-evaluating Claude judge — a cross-vendor judge is the obvious next control.)

Market triangulation (K) — June 2026 snapshot. (*Dated and treated as immutable per §10's moving-target caveat; from a multi-source landscape scan, confidence varying by source.*) The evidence corroborates P8a with unusual clarity:

- **The interop standards converge — but deliberately stop short of coordination state.** MCP (agent↔tool; donated to the Linux Foundation's Agentic AI Foundation, Dec 2025), A2A (agent↔agent; Google → Linux Foundation; *explicitly stateless* — discovery, delegation, message-passing, “without sharing internal memory, state, or tools”), and ACP (IBM; *merged into A2A* in 2025) are consolidating under neutral governance. None standardizes a durable, multi-vendor-readable *coordination record* (shared queue / role / lease / intent state); MCP's experimental Tasks primitive is a single-requestor call-now-fetch-later pattern, not shared state. Independent analyses and the protocols' own specs/roadmaps agree the coordination-state layer is the open, un-standardized slice.
- **Every first-party vendor coordination feature is single-vendor.** Claude Code's Agent Teams (file-based, local, Claude-only), OpenAI Codex sub-agents, Gemini/Jules, Cursor's multi-agent mode, Copilot's fleet, Amp, Devin-manages-Devin's — none lets a *different* vendor's agent join its coordination primitive. The host layer (e.g. VS Code's multi-agent sessions) runs rival agents side-by-side but with *no unified cross-vendor task list*. This is exactly P8a's prediction: vendors build coordination, and build it single-vendor.
- **The exact niche — a portable, program-owned, cross-vendor coordination record**

— **is occupied, and no longer only by small OSS.** GNAP (git-native agent protocol: roster / tasks / runs / messages as JSON in a plain git repo, “if it can git push, it can participate” — stalled since spring 2026), and the Beads + Gas Town pair (a DAG work-store plus a fleet orchestrator over git worktrees — whose OSS build went genuinely cross-vendor in June 2026, driving Claude Code, Codex, Gemini, and Copilot, with a federated cross-town work-exchange layered above; at roughly 24k/16k stars respectively, a serious occupant, not a curiosity). This both *validates the approach* and **temperers any uniqueness claim**: AIDA is not alone, git-native cross-vendor coordination is a small-but-real movement rather than a singular insight, and the honest comparison is against these neighbors, not a vacuum. (Beads notably moved its store to Dolt rather than plain git — a reminder that “git-canonical” is a deliberate choice, not the default.)

The pattern holds: capable teams keep building the lower layers and the single-vendor case; the neutral cross-vendor *record* is left to a handful of OSS projects — consistent with P8a’s incentive argument, though the handful now contains a well-distributed member, so the honest reading is “contested frontier,” not empty space. A mid-2026 corollary from the *hosting* lane points the same direction: cmux (~24k stars), a terminal purpose-built for running many vendor agents in parallel, validates strong operator demand for fleet-watching UX — panes, attention signals, programmable control — while deliberately shipping zero work-unit state (“a primitive, not a solution”). Even the distribution-rich neighbors keep stopping one layer below the coordination record. (Discovery-method finding, retained: we missed our nearest competitors for weeks by searching our *own* vocabulary inside a fixed category. Competitive discovery in a fast-moving field must be multi-modal — sweep awesome-lists, watch known builders, search the problem’s many names, follow star-velocity — not keyword-match your framing. A methodological contribution in its own right, and the reason GNAP/Gas Town above are in this draft at all.)

Implication for the build-vs-buy decision (not an AIDA pitch). For a large organization the question is not “which model” — that commoditizes — but “what owns the fleet.” As L0-L2 get absorbed upward into vendor platforms, L4-L5 is where a *home-grown* layer could still buy something a vendor will not provide: spanning the vendors an enterprise inevitably mixes, surviving any single vendor’s deprecations, and hosting governance/audit/policy that cannot be outsourced to a walled garden. Whether that justifies *building* it — versus waiting for a neutral incumbent, or accepting a vendor’s native coordination — is the open cost/benefit question of §12, not a foregone conclusion.

After red-team — split the claim. P8 as stated bundles a strong mechanism with a contingent business bet; the honest paper separates them and asserts only the first strongly:

- **P8a (strong, (M)):** a model vendor is disincentivized to build *portable cross-vendor coordination*, because portability dissolves the lock-in that within-vendor coordination exists to create. This is sound and is what the (M) grade covers.
- **P8b (contingent, (D)(K)):** therefore an independent, durable layer is defensible *by AIDA*. P8b does *not* follow from P8a. P8a establishes the gap exists; it does not establish a small player can hold it. P8b rests on three bets that must be named, not assumed: (i) vendor *heterogeneity persists* (enterprises do not consolidate on one vendor for procurement/governance simplicity); (ii) emerging interop standards do *not* commoditize the coordination *record* down to a thin protocol — *currently favorable* (as of June 2026 MCP/A2A/ACP have converged under the Linux Foundation but explicitly stop at tool-calling and delegation, leaving coordination state open), but the A2A↔MCP interop working group and NIST’s agent-standards initiative are the things to watch; (iii) AIDA *out-executes the non-vendor incumbents* — and as of mid-2026 they are **visibly moving on L5**: Microsoft’s Agent 365 “control plane” (with cross-vendor registry sync to AWS/Google agents in preview), Temporal positioning as an “agentic control plane,” control-plane startups being acquired, and OSS neighbors (GNAP, Gas Town) already in the exact niche. The red-team hole — *a neutral incumbent, not you, captures it* — is not hypothetical; it is materializing. The paper asserts P8a, argues P8b, and treats (i)–(iii) as the explicit,

falsifiable conditions the bet rests on. See §9.

A new-vendor onboarding-cost premise underneath P8b — now with pilot evidence. P8b’s business case quietly assumes a further, sharper sub-claim: that a *new vendor’s* agent can become productive in a running AIDA fleet at **near-zero onboarding cost** — because if joining the fleet costs a thick bespoke integration per vendor, the vendor-neutral layer is neutral in name only. This is the one thing single-vendor incumbents (Claude Agent Teams, Codex sub-agents) structurally *cannot* show: a rival vendor has no native seat in their coordination primitive at all. A cross-vendor portability pilot (ablations/2026-06-19-cross-vendor-portability.md, TASK-870) moved this from argument toward evidence. A genuinely different vendor (Codex, gpt-5.5), dropped into a throwaway live AIDA fleet with **nothing but the stock aida CLI on its PATH** and a one-line prompt (binary name + its agent name + the trailer convention), rode the full **discover → claim → ship → verify** loop — found its assigned brief on the *first* command, acked it, did the work, and shipped a (SPEC-ID)-trailed, auto-completed spec — in **~50 s with 0 lines of bespoke integration** (no wrapper, no MCP config, no glue). The credibility-raising part is that the probe *found and filed a bug* while passing: the brief’s generated **## Setup** block hardcoded the real repo’s path (BUG-583), which a cold vendor following it literally would have left the project on — Codex’s own judgment dodged it, and the defect is a concrete, fixable substrate finding rather than a hand-wave. **Honest scope: n=1 vendor, n=1 trivial task, CLI-only (no MCP leg).** It is a capability existence-proof for one vendor pair, not a study — the onboarding-cost premise holds for at least Claude→Codex; the apex bet still rests on the other P8b conditions (heterogeneity persists, interop doesn’t commoditize the layer, AIDA out-executes the incumbents), none of which a portability stopwatch can settle.

8. Orthogonal: the surface (P7)

P7 (Trojan-horse disclosure) — grade (D)(K). The weakest claim, and the one most about the artifact rather than the problem. We record it as an *observation about home-grown-tool adoption*, not a recommendation. For a tool whose value is *emergent* (a graph, a coordination fabric), a deliberately shallow surface that defers depth-disclosure to use seemed to land better than one advertising its depth — the “I could do this in 20 lines of bash” first reaction was acceptable because the value (IDs, graph, traces, the merge model, the cascade) is only legible after living in it. We flag it precisely because it is *least* generalizable: it may simply describe one operator’s taste, and it is the finding most entangled with promoting the artifact — which this paper explicitly is not trying to do.

P7b (the agent-facing surface, measured) — grade (D). The surface question has an agent-side half that *did* survive controlled measurement, and the result inverts a common intuition. A 72-cell benchmark (SPIKE-73: 4 surfaces × 6 tasks × 3 runs, deterministic grading; bench/agent-surface/results/report.md) priced the same substrate reads over four surfaces: the plain human-formatted CLI (100% success, ~\$0.036/task), MCP (89% success, **~2× the cost**), MCP with on-demand schema loading (100%, still ~1.8× — the extra schema round-trips eat the input-token savings and turns/tools *rise*), and a token-efficient CLI encoding (parity on small reads; 21-84% cheaper on large list/show outputs, measured separately). The intuition that a typed, structured protocol is the “highest-leverage” agent interface does not survive contact: the premium is structural, not a schema-loading artifact. The problem-space finding: **agents do not need your typed protocol; they need cheap, stable text.** A typed surface is worth *offering* (some clients only speak MCP), not *defaulting*. This also scopes P2’s mechanism paragraph honestly — the typed graph pays (P2b, relational/rename), but the economical access path to it is the CLI, not the protocol that seemed to be the point of having a graph.

9. Strongest objections and restatements

We red-teamed the two load-bearing propositions — P3 (the most counter-intuitive) and P8 (the apex bet) — and let the strongest objections rewrite the claims. The result is two precision edits: both *narrow* the assertion to ground we can defend, surrendering the indefensible part before a reviewer takes it. This is the paper’s own stance applied to itself (state the precise open slice; do not overclaim).

P3 — substrate-bounded autonomy

Objection	Bite
Cold-boot is self-imposed	Each resolution is a cold boot only <i>because</i> AIDA spawns a fresh claude -p; fork-from-live and a persistent advisor (both on the roadmap) dissolve the premise. The trend in vendor platforms (persistent memory, long sessions) is <i>against</i> the premise.
Substrate $\perp$ capability is false	A more capable model needs <i>less</i> substrate for the same decision, so the two trade off and cannot be varied independently; the effect likely shrinks as models improve.
Difficulty confound	Richer substrate correlates with better-specified (intrinsically easier) specs, so an ablation risks measuring problem difficulty, not substrate.
Necessary $\neq$ dominant	Substrate may set the <i>floor</i> while the model sets the <i>ceiling</i> ; “substrate is necessary” is not “substrate is the binding constraint.”

Restatement (asserted): *Under stateless escalation, substrate enrichment is the cheapest, most reliable autonomy lever.* The unscoped “substrate dominates capability” form is held only weakly and time-bounded to the current model tier.

P8 — incentive-persistent cross-vendor gap

Objection	Bite
Newness, not incentive	The gap may simply be young and close on its own; “structural persistence” cannot yet be distinguished from immaturity.
A gap AIDA can’t hold	Even if no <i>model vendor</i> builds L5, GitHub/Microsoft, the IDEs, LangChain, and “agent control plane” startups are equally incentivized and have distribution AIDA lacks. The gap existing $\neq$ a small player capturing it. (Largest hole.)
Interop commoditizes the layer	MCP / A2A / ACP show vendors <i>do</i> have interop incentives; if coordination standardizes at the protocol layer, a mailbox/roles/leases substrate is standardized away.

Objection	Bite
Multi-vendor is transient	Enterprises may consolidate on one vendor for procurement/governance simplicity, shrinking the multi-vendor premise.
Native > neutral on UX	A neutral layer is often lowest-common-denominator; users may prefer a vendor's better-integrated native coordination.
Self-serving framing	P8 is the claim most aligned with "therefore build AIDA" — exactly the one its authors should most distrust (the §10 self-evaluation threat bites hardest here).

Restatement (split): assert **P8a** (vendors are disincentivized to build *portable* coordination) strongly on the (M) grade; argue **P8b** (an independent durable layer is defensible *by AIDA*) as contingent on three named, falsifiable bets — heterogeneity persists, interop does not commoditize the layer, AIDA out-executes the non-vendor incumbents. The business case lives entirely in P8b and its three conditions, not in P8a.

10. Threats to validity

- **N=1 system, N≈1 operator.** Every (D) claim is autoethnographic. We cannot separate "AIDA's design helped" from "writing intent down at all helped," nor from "this particular operator's discipline helped."
- **Alpha maturity.** The system is pre-1.0; some mechanisms (e.g., the cold-boot advisor) are recent and lightly exercised.
- **Self-evaluation.** The authors designed the propositions and the system; confirmation bias is unmitigated by independent review.
- **Market grade (K) is observational.** Competitor behavior is consistent with P8 but does not prove the incentive mechanism; an alternative explanation (the gap is simply newer) cannot yet be excluded.
- **Moving target.** The stack's value distribution (§1) is shifting during the study; any snapshot of "how far up value has moved" dates quickly. We treat dated artifacts as immutable observations, not standing claims.

11. Faulty assumptions this probe falsified

The clearest argument for *building* rather than *analyzing* is the set of beliefs that fail only once the system is real. Each below was held at some point and falsified by the dogfood; several became the findings above. They cluster in two places — the gap between *what we assumed an LLM would do* and what it did, and the unglamorous infrastructure seams — and neither is visible from analysis alone.

Assumption (believed)	What building falsified it	Becomes
Rules and prompts can govern a capable agent	Agents routed around CLAUDE.md / memory rules confidently on <i>semantic, multi-step, far-from-the-action</i> invariants; only programmatic gates held the line there	P1 (substrate-as-bouncer)

Assumption (believed)	What building falsified it	Becomes
...and the converse: <i>a stated rule is insufficient against a capable agent</i> (the blanket form)	Falsified across every controlled condition we could construct — rule-only compliance was 100% (10/10) on an <i>immediate</i> invariant (I1), 100% on a <i>buried, semantic</i> one (I2-Claude), 100% again with Codex (I2 cross-vendor), 100% on a <i>controlled procedural/tool-use</i> one (I3), and 100% on a <i>complex multi-file</i> procedural task with the rule buried among ~8 instructions (I4); the gate did no work in any. <i>Five</i> single-variable conjectures were pre-registered and falsified in sequence — <i>rules-just-fail</i> (I1), <i>attention-distance</i> (I2), <i>vendor</i> (cross-vendor I2), <i>invariant-type</i> (I3), <i>task-complexity</i> (I4) — each drawn through the one uncontrolled bake-off drop. With five controlled cells all at the ceiling, the verdict is definitive and methodological: a clean ablation cannot reproduce rule-dropping at all. The lone drop lives in the unmeasured real-repo/long-autonomy regime a controlled design cannot reach; the open path is field instrumentation (count rule-drops in real drains), not a sixth ablation — whose first two slices have since run (SPIKE-67) and returned a first-repo null: the span effect washes out under a commit-type control, and drained commits adhere <i>better</i> (2%) than interactive (17%)	P1 (terminus: rules suffice across every controlled condition; field slices 1-2 agree on this repo — the gate's case survives only on undisciplined external histories, pending the cross-repo harvest)

Assumption (believed)	What building falsified it	Becomes
Open briefs yield divergent designs (so multi-vendor competition buys architectural diversity)	Falsified — two vendors on a deliberately open brief converged on the same command, axes, module, and verdict model; convergence is <i>substrate-driven</i> , and divergence survives only in the residual the substrate leaves unspecified	P2-strengthened / competition-as-QA (§3, §7)
A more capable model reduces the need for coordination infrastructure	The hardest autonomy failures were context-starvation at cold-boot, not reasoning limits; a bigger model does not help a fresh process that lacks the substrate	P3 (scoped)
A git-backed multi-writer store means merge hell	False for the <i>append-union</i> half (G-Set by UUID — genuinely conflict-free); but the scalar half is wall-clock LWW with an “ours” tie-break, non-deterministic across clones — the HLC that was supposed to make it deterministic is built but unwired (BUG-578, found by red-team). “Few textual conflicts” $\neq$ “deterministic scalar convergence”	P4 (split: P4a append-union holds; P4b HLC-determinism rebutted)
Requirements tracking can be bolted on after the code	ID instability and missing traces made retrofitting the graph expensive; it must be load-bearing from the first commit or agents cannot name things unambiguously	P2a (the stable name is the floor)

Assumption (believed)	What building falsified it	Becomes
The rich intent graph (typed relationships, traces, MCP) clearly pays for its upkeep	Half shown, controlled (RYO benchmark, TASK-871): same specs at 10/50/200/500 through markdown+git+grep vs. an aida store, deterministic grading — the graph <i>pays</i> on relational queries ($\sim 80\times$ at 500, flat vs. grep’s linear climb) and on rename blast-radius (RYO silently rots 2 refs at every scale ≥ 50 ; AIDA id-stable, 0 missed), while grep <i>wins</i> on full-text + flat-list. So “dead weight” is falsified for relational/rename work, conceded for textual work — capability-friction only; the \$10 team-adoption confound is named, not escaped. P2a (stable name) holds; the two red-team undercuts (convergence isolates physical shape; traces rot invisibly) still bound it	P2 (split: P2a holds (M); P2b — relational/rename half now controlled, team-adoption half still open (D))
The calibration loop self-corrects future estimates	Only 2/3 built (red-team, TASK-846): record + compare exist (complexity_calibration.rs/effort_calibration.rs/calibration.rs → findings_calibration --stats), but no code tunes a future estimate — read-back ends in a report a human reads. AIDA ships a calibration <i>dashboard</i> , not a self-correcting <i>estimator</i> ; the “60%-learner beats static-80%” figure is illustrative, never measured	P5 (split: P5a record+compare hold; P5b strong form is conjecture)
Cross-vendor coordination is a capability problem the next model solves	It is an incentive problem; no model release moves it	P8a

Assumption (believed)	What building falsified it	Becomes
Onboarding a <i>new vendor</i> into the fleet costs a per-vendor bespoke integration	Pilot evidence against (portability test, TASK-870): a cold Codex (gpt-5.5), given only the stock aida CLI + a one-line prompt, discovered → claimed → shipped → verified a (SPEC-ID)-trailed auto-completed spec in ~50 s with 0 lines of integration — and surfaced + filed a brief-template path bug (BUG-583) en route. n=1 vendor, n=1 trivial task, CLI-only; an existence-proof, not a study	P8b (onboarding-cost sub-claim — near-zero for Claude↔Codex)
Scaffolding / init is harmless and idempotent	An aida init on a fresh clone silently committed a regenerable scaffold dump and polluted the shared main branch (BUG-570, found mid-writing). Home-grown infra has minefields exactly where attention lapses	a direct cost-of-RYO datapoint (\$12)
I can find my competitors by searching my own framing	We missed our two nearest competitors for weeks; discovery had to go multi-modal — a faulty assumption about the <i>research method itself</i> , not the product	a method finding
A shared store with a user field gives coherent team identity for free	Identity fragmented across three spaces — the integer node user_id, the shell \$USER string, and email — so the team dashboard grouped on one key while roles enforced on another; a role set in the UI silently never applied. Identity is a substrate problem, not a field (found this session)	a coordination finding (L2/L4)
A git-canonical store with a thin web reader scales for reads	The reader loaded the full graph (~6.5 MB of nested history/comments) on every view and stalled the UI at a few hundred specs; reads need a summary projection + detail-on-demand. The write model and the read model are different problems (found this session)	a direct cost-of-RYO datapoint (\$12)

Assumption (believed)	What building falsified it	Becomes
Flagging a spec “needs a human” is enough to route it to a human	An <i>ungroomed</i> draft marked human-only surfaced as a “decision” with no question to answer — you cannot decide on something ungroomed. The operator/advisor seat split is grooming-gated, which a naive flag bypasses (BUG-572, found this session)	a coordination finding (L4): routing is grooming-gated

The pattern is the justification for the probe: these were not knowable by reasoning about the problem; they required walking the ground. That value accrues *whether or not the artifact survives*.

12. The cost/benefit of roll-your-own (verdict: open)

We state plainly that we do **not** have a verdict on whether a home-grown coordination layer is worth building. The probe is still measuring. Here is the ledger as it stands.

Costs (observed). - *Churn against shifting vendor surfaces.* During the build, vendor-native primitives (sub-agents / agent teams, MCP, skills, project memory) appeared and moved *under* the home-grown layer, repeatedly turning built features into redundant or competing ones. You build on sand, and the sand is being poured by better-resourced teams. - *The unglamorous substrate tax.* An ID dispenser, a hybrid logical clock, a structured three-way merge driver, role-gating, cross-platform CI debt, scaffolding/init correctness (BUG-570) — none of it the interesting part, all of it required before the interesting part works. - *The dogfood maintenance tax.* The system must work well enough to build itself, every day, before it can teach you anything. - *The fleet’s operating bill — now measured, and movable* (2026-06-29-agent-fleet-economics.md). Running the probe priced its own operation on two axes. Tokens: the typed/MCP agent surface costs $\sim 2\times$ the plain CLI on identical tasks (so the token-efficient CLI became the default surface), and timer-based LLM supervision of an overnight drain burned an estimated \$6–20 in wakes that found nothing until replaced by event-driven watching (a cheap non-LLM classifier absorbs benign events; the LLM wakes on actionable verbs only — O(real events), zero tokens while silent). Compute: a cold worktree per fanned-out agent pays a $\sim 30\times$ avoidable build tax that a warm recycled pool reclaims (5.5 s $\rightarrow$ 182 ms per reuse on the pilot crate; a real workspace’s delta is larger). None of these costs is unique to roll-your-own — but roll-your-own means *you* discover and fix each one.

Benefits (observed). - *Control of the bouncer.* You can place gates and invariants (P1) exactly where a vendor will not, because the incentives differ — but the gate-vs-rule ablations *narrow* the demonstrated value sharply, and honesty requires saying how. Five controlled cells — output-shape and procedural, across both vendors, near and far from the action, trivial through complex — *all* held a stated rule at the ceiling with the gate idle, so across **everything controllably measurable the evidence says a gate buys nothing** and gating there is surface bloat. The program reached its terminus: a clean ablation cannot reproduce rule-dropping at all, so it cannot say which invariants need gates by experiment. The one observed rule-drop was a **complex, real-codebase, long-autonomy** task that no controlled design can reach, and AIDA’s actual hard gates (the drain rails, don’t-merge-over-RequestChanges, push-and-PR atomicity) all live in exactly that regime — so the evidence neither confirms nor refutes them; their justification lives in a regime measurable only by **field instrumentation** (counting rule-drops in real drains), not by more ablations — and the first two field slices (SPIKE-67) now return a null *on this repo’s own history* (drained commits adhere best of all cuts), so the

gate’s remaining case rests entirely on undisciplined external repos until the cross-repo harvest runs. The defensible benefit, then, is *control of where to gate* — with the discipline to **not** gate rules that only ever fire in small, well-scoped tasks, and to **audit** the complex-regime gates against the real-repo bar rather than treat them as vindicated. - *Vendor-neutrality and durability*. Nothing in the substrate is hostage to one vendor’s roadmap or deprecations. - *Shaping the fleet’s output*. Owning the substrate does not merely lower coordination cost — it determines the *shape* of what every vendor’s agent builds (§3, §7). That is a benefit no bought or borrowed layer confers. - *The learning itself*. You cannot map the minefield (§11) without walking it. Much of this paper did not exist as analyzable knowledge until the system forced it into the open. If the goal is *understanding the problem*, building is a uniquely high-bandwidth instrument — even if the artifact is later discarded.

Compose instead of own? A natural cost-reducer is to *compose on a neighbor* — sit AIDA’s model on top of Beads’ store via a backend adapter — rather than own the store. We probed it (2026-06-17-aida-on-beads.md, SPIKE-65) and the honest finding tempers the verdict in a specific direction: **composing trades one lock-in for another and adds a thick impedance layer; it does not reduce cost**. Beads is a different *model*, not just a different store — its 3-state unenforced lifecycle cannot carry AIDA’s approval gate, its event-level history cannot reconstruct per-field audit, and code-to-spec traces have *no* equivalent — so a BeadsBackend is a ~3-4× thicker translation layer than a plain SQLite backend, coupled to a fast-churning Dolt schema. Composing swaps AIDA’s inspectable git-YAML lock-in for Beads/Dolt lock-in (SQL engine required) *plus* a schema-churn maintenance tax. What owning the store actually buys is therefore **portability + inspectability, not lower cost** — and the only uncontested differentiator that survives the round-trip is the code-to-spec trace (which itself requires the enforced lifecycle Beads lacks). The right “compose” move is the reverse: a one-way Beads→AIDA importer that lets a neighbor’s users opt *into* the governance layer, not AIDA inheriting their store.

The asymmetry that shapes the (still-open) verdict. Cost and benefit are *layer-dependent*. Lower-stack cost (L0-L2) is being eroded as vendors solve it for free, so building there increasingly duplicates work that arrives anyway. Upper-stack (L4-L5) is where home-grown still uniquely buys something — but it is also where the largest red-team hole sits (a neutral incumbent, not you, may capture it; §9, P8b). So the honest lean, held loosely, is *against* rolling your own at the bottom and *toward at-most-experimenting* at the top — but we assert neither as a conclusion.

What now has experimental support vs what remains argument. The first round of experiments has begun to move the ledger, and honesty requires marking which sub-claims have shifted — including the one that shifted *against* an earlier reading. *Now with ablation evidence*: the **strong negative on P1** (the gate-vs-rule pilot plus I2, cross-vendor I2, I3, and I4, deterministic grading — five controlled cells, all at the ceiling, which together *falsified* five pre-registered single-variable conjectures in sequence: rules-just-fail, attention-distance, vendor, invariant-type, and task-complexity). The surviving P1 claim is narrow and methodological: *rules suffice across every controlled condition; the gate does no work there; a clean ablation cannot reproduce rule-dropping at all, so it cannot identify when a gate would win*. The **substrate-shapes-output** reading of P2/L5 (the open-brief convergence run) also has ablation support. *Two further claims moved this round*. The **P2b “does the rich graph pay” question** — the verdict’s biggest open question on the intent layer — now has its first controlled cut (RYO friction benchmark, TASK-871): the typed graph *pays* on relational queries (~80× latency at 500 specs) and rename blast-radius (silent ref-rot prevented at every scale ≥ 50), *grep wins* on full-text + flat-list, so the “dead weight” reading is falsified for the relational/rename half and conceded for the textual half — capability-friction only, the team-adoption confound named not escaped. And the **P8b onboarding-cost sub-claim** has its first datapoint (cross-vendor portability pilot, TASK-870): a cold Codex joined a live fleet with 0 lines of integration and shipped a trailered auto-completed spec in ~50 s — the one thing single-vendor incumbents structurally cannot demonstrate — though n=1 vendor on one trivial CLI-only task. *Now with a structured*

negative-result method: multi-vendor competition reframed as a QA mechanism, and compose-on-a-neighbor priced honestly (SPIKE-65). *Still argument, not evidence*: when a gate beats a rule — the field slices (SPIKE-67) have since run and returned a first-repo null that *agrees with* the lab (span effect washes out under type control; drained commits adhere better than interactive), so the question now lives exclusively in the cross-repo harvest over less-disciplined histories; **whether the rich graph pays once a team (not one disciplined operator) maintains the cross-references** — the half of P2b the benchmark cannot reach; and the apex bet itself: P8a (incentive-persistence) rests on (M) + market observation, and P8b’s three conditions (heterogeneity persists, interop doesn’t commoditize the layer, AIDA out-executes the incumbents) remain unsettled — the portability pilot moves only the onboarding-cost premise *underneath* them, not the incumbent-capture hole. The verdict on roll-your-own is therefore **open but better-instrumented**: the lower-stack lean (*against* — vendors absorb it) and the upper-stack lean (*toward at-most-experimenting* — where the incumbent-capture hole sits) both stand, now with the added, evidence-backed refinements that *the substrate is the lever* the whole bet turns on, that *the graph earns its keep on relational/rename work but not on text*, and that *a new vendor’s onboarding cost can genuinely be zero*.

What would settle it. The gate-vs-rule program has reached its terminus — *I4 ran and weakened the last candidate (complexity), so all five controlled cells are at the ceiling and a clean ablation cannot reproduce the drop; the open path is **field instrumentation**, not more ablations* — so what remains to settle the broader claims is the §13 *cross-repo* field harvest (the first two single-repo slices are in and returned a controlled null — the effect must be found on undisciplined external histories or the gate case closes), the *team-adoption* half of the rich-graph question the RYO benchmark could not reach (the benchmark settled capability-friction; a multi-team study settles whether the graph still pays once humans, not one disciplined operator, maintain the cross-references), the *remaining rows* of the cross-vendor portability test (TASK-870 ran the first — Claude↔Codex, zero-integration, ~50 s; a 4+ vendor table + a real-implementation task remain), and the multi-team field study — plus the simple passage of time: watch whether vendor surfaces converge on the upper layers within roughly a year. Until then, the verdict is open, and saying so is the finding.

A companion decision framework — 2026-06-16-layered-evaluation-framework.md — operationalizes this open verdict into a per-layer **build / buy / ride-native / wait** rubric with lock-in safeguards, a cross-cutting scoring checklist, and the *substrate-as-organizational-asset* thesis (the shared intent+coordination graph as a compounding, non-purchasable competitive advantage).

13. What a real study would measure

Red-team provenance. P2, P4, and P5 received a dedicated adversarial red-team pass (TASK-846) — P4 verified directly against code, yielding BUG-578 (the HLC is built but unwired; scalar merge is wall-clock LWV). **P1, P3, and P8 were already adversarially scoped** (the gate-vs-rule ablation program for P1; the §9 objection tables for P3 and P8). The TASK-846 pass was **within-vendor** (Claude red-teaming a Claude-authored paper) with deliberately adversarial framing — so it shares the §10 self-evaluation threat: a same-vendor critic may miss blind spots a different vendor would catch. A cross-vendor red-team is the obvious next control.

To move claims from (M)(D) to confirmed empirical results:

1. **Multi-team, multi-vendor field study.** Several teams, each running a mixed Claude/Codex/Gemini fleet, half with AIDA’s substrate and half with unstructured markdown+git. Measure rework rate, “done-but-not-in-diff” incidence, time-to-context for a cold-boot agent, and merge-conflict rate on shared state.
2. **Gate vs rule ablation (P1) — program complete (terminus).** The same invariant enforced as a CLAUDE.md rule vs a programmatic gate; measure violation rate per

- 100 agent-actions. *Piloted five times, all at the ceiling*: an immediate format invariant (2026-06-18-gate-vs-rule-pilot.md, 100% rule-only — falsifying *rules-just-fail*); a buried, semantic, far-from-the-action invariant (2026-06-18-gate-vs-rule-i2.md, n=10/arm, 100% rule-only, 0 gate-saves — falsifying *attention-distance*); that same buried invariant re-run with **Codex** (cross-vendor I2, 100% rule-only, 0 gate-saves — falsifying *vendor*); a controlled *procedural/tool-use* invariant (2026-06-18-gate-vs-rule-i3.md, n=10/arm, 100% rule-only, 0 gate-saves — falsifying *invariant-type*); and a *complex* multi-file/multi-function procedural task with the rule buried among ~8 ambient instructions (2026-06-18-gate-vs-rule-i4.md, Claude n=10/arm, 100% rule-only, 0 gate-saves — falsifying *task-complexity*). Five controlled cells, every one at the ceiling, five single-variable conjectures falsified in sequence → the verdict is definitive and methodological: **a clean ablation cannot reproduce rule-dropping at all**. The lone observed drop (the bake-off, n=1) lives in the unmeasured real-repo/long-autonomy regime a controlled design cannot reach. **The open path is no longer a sixth ablation — it would hit the same ceiling — but field instrumentation**: count stated-rule violations in real aida queue work --auto-complete drains on real repos and correlate with repo size / task span / context pressure. That is the remaining decisive evidence, and a different *kind* than this program could produce. *First two slices run (SPIKE-67, 2026-06-20-... + 2026-06-22-field-study-slice2-...)*: the scanner treats the git log as the already-planted sensor; on this repo (750 commits) the effect **vanishes under controls** — a squash-merge suffix had inflated the raw rate ~2.5×, holding commit type to feat/fix flattens the span curve, and drained commits adhere better (2%) than interactive (17%). A valid null, converging with the lab; it also delivered a cautionary methods tale (the drain axis *reversed* once the artifact was stripped). Remaining: the cross-repo harvest over less-disciplined histories and session-level (headless-vs-supervised) attribution.
3. **Rich-graph-vs-markdown ablation (P2b) — first controlled cut run**. Two arms, same specs and same operator: typed graph + traces vs plain markdown + git + grep. *Piloted (2026-06-19-ryo-friction-benchmark.md, TASK-871, deterministic grading, 10/50/200/500 specs, five tasks incl. two grep-favoring fairness tasks)*: the graph **pays** on relational queries (~80× latency at 500, flat vs. grep’s linear climb) and rename blast-radius (RYO silently rots 2 refs at every scale ≥ 50; AIDA id-stable, 0 missed) and grep **wins** on full-text + flat-list — so “dead weight” is falsified for the relational/rename half, conceded for the textual half. This is **capability-friction**; what it cannot reach is the **team-adoption confound** — whether a *team* (not one disciplined operator) keeps markdown cross-references consistent by hand, which is exactly the discipline the graph automates. The decisive remaining test therefore measures coordination quality across teams *after* removing the operator-discipline confound. A cheaper internal proxy: a **trace-read-rate audit** — query_graph/show_requirement read volume against trace-write volume; written-but-not-consulted falsifies P2b’s value.
 4. **Substrate-richness ablation (P3)**. Hold the model fixed; vary only how much context the cold-boot advisor’s substrate carries; measure resolution quality. P3 predicts substrate dominates model tier.
 5. **Scalar-merge determinism repro (P4b / BUG-578)**. Two clones merge a pair of concurrent *same-field* scalar edits in opposite roles; the rebuttal predicts they converge to **different** winners depending on which clone runs the merge (last-puller-wins) — and that clock skew lets the wall-clock-later edit beat the causally-later one. A passing determinism test requires wiring the HLC (or a data-function tie-break) into conflict.rs.
 6. **Learner-vs-static calibration ablation (P5b)**. The decisive test the strong form needs: **two arms with the base model held fixed** — one consuming prior calibration records to adjust the next estimate (the unbuilt *tune* leg), one static — measured to a *crossover* where the learner overtakes. Until that ablation runs, the “60%-learner beats static-80%” figure is illustrative, not measured; note that the subjective ground truth means even a positive result shows convergence to *agreement*, not correctness, and the non-stationary estimand needs a stationarity argument.
 7. **Cross-vendor portability test (P8b) — first row run**. Introduce a new vendor agent

into a running fleet with zero bespoke integration; measure time-to-productive. P8b’s onboarding-cost premise rests on this being near-zero. *Piloted (2026-06-19-cross-vendor-portability.md, TASK-870, deterministic grading)*: a cold Codex (gpt-5.5), given only the stock aida CLI + a one-line prompt, discovered → claimed → shipped → verified a (SPEC-ID)-trailed auto-completed spec in **~50 s with 0 lines of integration**, and surfaced + filed a brief-template path bug (BUG-583) en route. **n=1 vendor, n=1 trivial task, CLI-only (no MCP leg)** — an existence-proof for one vendor pair. The decisive version is the 4+ vendor (Gemini/Jules/Cursor/...) time-to-productive table plus a real-implementation task and an MCP-surface leg.

14. Related work

Foundations. The conflict-light store (§5) is an applied instance of CRDTs (Shapiro et al.) — G-Sets, LWW-Registers, OR-Sets — embedded in git rather than a bespoke runtime. The method frame is design-science research (Hevner) and the software-engineering *experience-report* tradition. The shared-store-plus-messages coordination pattern is a modern, vendor-neutral *black-board* architecture. The floor it builds on is Karpathy-style “structured markdown as context”; AIDA’s claim is that the typed graph + ID stability + enforcement loop is the layer *above* that floor.

Interop standards (June 2026). MCP (agent↔tool, under the Linux Foundation’s Agentic AI Foundation), A2A (agent↔agent discovery/delegation, explicitly stateless), and ACP (merged into A2A) standardize how agents call tools and pass messages — *not* a shared coordination record. They are the protocol layer *beneath* the coordination-state layer this paper concerns (see §7).

Spec-structuring neighbors. GitHub spec-kit (vendor-neutral, git-portable, but sequences a single agent), AWS Kiro (single-vendor, spec-driven IDE), OpenSpec, Backlog.md — these structure specs/tasks but do not coordinate fleets; they are L1/L3 neighbors, not L5.

Nearest neighbors — portable cross-vendor coordination. **GNAP** (git-native agent protocol: roster/tasks/runs/messages as JSON in plain git; cross-vendor; no lease mechanism) and the **Beads + Gas Town** pair (a DAG work-store plus a fleet orchestrator over git worktrees; Beads now Dolt-backed) occupy AIDA’s exact niche and are the most direct comparison. AIDA’s distinguishing choices are the typed *relationship graph* (vs flat issues/tasks), lease/role gating, and code-to-spec traces — but the family resemblance is real, and the paper’s claims should be read against these, not in a vacuum.

Orchestration / control planes. LangGraph, CrewAI, the Microsoft Agent Framework, Temporal, and Microsoft Agent 365 orchestrate or govern multi-agent work but hold coordination state in a service or process, not a portable program-owned record. They are the L4 orchestration neighbors and, increasingly (§9, P8b-iii), the incumbents most likely to contest L5.

Execution orchestrators. A large class of parallel-execution tools — Claude Squad, Conductor Build, Code Conductor, Axel, SPECTRE, and others — fan agents out across isolated git worktrees with a review/merge dashboard. Of nine surveyed (June 2026), eight maintain *no* durable coordination model; they are L4 execution fan-out, not L1/L3/L5. *graft* is a related but distinct runtime lock-manager — OS-style claim-before-write leases at file-resource granularity (state in an in-memory bus), preventing concurrent-edit conflicts — complementary to intent coordination, not a substitute for it.

Identity and authorization. *Authenticated Delegation* (South et al., arXiv:2501.09674, 2025) extends OAuth/OIDC with agent-ID and delegation tokens so a third party can verify an agent acts for a named principal within granted permissions. It is a trust substrate a coordination layer can sit *atop* — complementary, and a natural pairing for a cross-vendor fleet that must prove who authorized what.

References

Formal citations for the foundational scholarly works invoked above (verified June 2026). The tool and standard *neighbors* named inline (spec-kit, Kiro, OpenSpec, Backlog.md, GNAP, Beads, Gas Town, graft, LangGraph, CrewAI, the Microsoft Agent Framework / Agent 365, Temporal, Claude Squad, Conductor, and the rest) are living projects cited by name; their canonical sources are their own repositories and spec sites, and the dated landscape survey behind them is docs/competitive-analysis/ (a deliberately living, dated artifact, not a frozen bibliography). The interop standards — MCP (Model Context Protocol), A2A (Agent-to-Agent), and ACP (merged into A2A) — are cited to their published specifications.

1. Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *Conflict-Free Replicated Data Types*. In Proc. 13th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems (SSS 2011), LNCS 6976, pp. 386–400. Springer. (Extended as INRIA Research Report RR-7506, *A Comprehensive Study of Convergent and Commutative Replicated Data Types*, 2011.) — the G-Set / LWW-Register / OR-Set foundations §5 applies in git.
2. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). *Design Science in Information Systems Research*. MIS Quarterly, 28(1), 75–105. — the design-science method frame (build an artifact as a probe to produce knowledge) this paper operates under.
3. Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). *The Hearsay-II Speech-Understanding System: Integrating Knowledge to Resolve Uncertainty*. ACM Computing Surveys, 12(2), 213–253. — the original blackboard architecture; AIDA's shared-store-plus-messages coordination is a modern, vendor-neutral blackboard.
4. South, T., Marro, S., Hardjono, T., Mahari, R., Whitney, C. D., Greenwood, D., Chan, A., & Pentland, A. (2025). *Authenticated Delegation and Authorized AI Agents*. arXiv:2501.09674. — the OAuth/OIDC-extending trust substrate a cross-vendor coordination layer can sit atop.

(The “structured markdown as context” floor AIDA builds above is attributed to A. Karpathy’s informal public commentary, not a formal publication.)

15. What the probe produced — distinctive value, honestly assessed

The probe stance forces a specific question: not “is AIDA good,” but *what value, if any, did building it encapsulate that is genuinely distinctive rather than commodity — and how much of that survives contact with the current landscape (June 2026)?* We sort by defensibility and concede where neighbors match, because conceding is the point. We also count, alongside the artifact’s distinctive choices, two artifact-independent outputs: the transferable findings (Tier 4) and the **experimental program + method** the probe produced this round (Tier 4b) — a thing the probe makes, not just a property of the tool.

The honest correction, first. Our initial read was that AIDA *fuses* a typed relationship graph + code-to-spec traces + lifecycle gates into a coordination substrate no neighbor holds whole. A landscape scan falsifies the strong form: the **Gas Town / Beads stack** (Yegge) holds most of that fusion — Beads is a typed-relationship issue graph (relates_to / duplicates / supersedes / parent-child + blocker-aware “ready” detection; ~24.6k stars at scan time), Gas Town adds seven coordination roles and enforced merge gates (the *Refinery* queue), it is cross-vendor and durable. AIDA is therefore **not unique in the fusion**; it is one member of a small family — with Gas Town/Beads the *more mature* member by traction — distinguished only by narrower choices. (Dated snapshot, treated as immutable per §10.)

A caution against overselling the sibling. Traction must be read carefully — stars reflect hype as much as engineering (“era of vibe”). Beads’ storage history also resolves an apparent

conflict in the reporting: it ran a *SQLite-database-synced-to-JSONL-by-a-background-daemon* design through early 2026, then removed SQLite (v0.58, Mar 2026) and made **Dolt** (a versioned SQL database) the source of truth (v1.0.5, May 2026), with JSONL demoted to an optional, non-canonical export. So “Beads is git-native” is now *stale*, and the sharpest peer criticism (auto-daemon, dual-store sync, schema churn, version-migration breakage) largely describes the *superseded SQLite era* — cite it as history, not current state, or it oversells the flaw as much as the stars oversell the tool. The durable contrast survives the correction: AIDA’s single-source git-YAML store (plain files, a *rebuildable* cache, no daemon, no SQL engine) is a deliberate alternative to *both* of Beads’ eras — not a less-mature version of either.

Tier 1 — what survives as genuinely distinctive (narrowed). - **Code-to-spec inline traceability** (`// trace:SPEC-ID`, machine-checkable from either direction): found in *none* of the dozen-odd tools surveyed (Gas Town only indirectly, via issue IDs in commits). The satisfaction relation living in the source itself is the sharpest surviving differentiator. - **Git-YAML-canonical store** (one plain YAML file per spec; SQLite a rebuildable cache) vs the family’s *database-as-source-of-truth* (Beads requires Dolt). A real architectural fork: AIDA’s record is readable with git and a text editor, works anywhere git works, needs no database engine — the “program-owned, inspectable, portable” property in its strongest form. - **Spec-lifecycle authority gates** — who may advance Draft→Approved→... (advisor-only transitions, merge-driven completion, trailer auto-complete) — as distinct from merge-time *verification* gates (Refinery’s CI queue). AIDA gates the *authority to change intent state*; the family gates *what merges*. Different, rarer enforcement point. - **The MCP requirement-graph surface** (~60 tools exposing the typed graph to any agent): not matched by the fan-out tools; Beads exposes a CLI/SQL graph of a different shape.

Tier 2 — distinctive but contested (the family holds it too). The typed relationship graph itself, role/lease coordination, and durable cross-vendor coordination are all present in Beads/Gas Town and *more mature there*. AIDA’s versions differ in shape — a typed *requirement* graph (19 node types) vs an issue graph; an advisor/implementer/reviewer/integrator *seat* model vs Mayor/Polecat/Refinery — but the category is shared, not owned. And the “typed graph + stable IDs” claim is *weaker still* than the OSS comparison suggests: mature, API-queryable typed-relationship graphs also ship in **Linear** and **Jira/Rovo** (Jira’s link graph pre-dates its agents by ~15 years). So “we have a graph, they don’t” is simply false; the only defensible narrower claim is a *git-portable* typed graph with *code-to-spec traceback* layered on it — which is Tier 1, not this tier. The one wedge that holds against *almost everyone* (trackers delegate one issue to one agent) is the **fleet-coordination substrate** — queue / lease / drain / escalation / inter-agent mailbox — matched only by Beads (DB-backed) and the stalled file-based farol-team/GNAP.

Tier 3 — commodity (where most of the field lives). Parallel execution + worktree isolation + dashboard (Claude Squad, Conductor Build, Code Conductor, Axel — 8 of 9 surveyed); spec-structuring as portable markdown (Kiro, SPECTRE, spec-kit); runtime file-lock arbitration (graft). Useful, replicable, not where durable value sits.

Tier 4 — the value that does not depend on the artifact. The transferable knowledge (P1-P8). Even granting that Gas Town/Beads matches much of AIDA’s surface, the *findings* — substrate-as-bouncer (now at its terminus: five controlled cells all at the ceiling, after I2 falsified attention-distance, cross-vendor I2 falsified vendor, I3 falsified invariant-type, and I4 weakened the last candidate, task-complexity — so a clean ablation cannot reproduce rule-dropping at all, and “when a gate beats a rule” is answerable only by field instrumentation, not more ablations), substrate-bounded autonomy, the conflict-light *append-union* recipe (its scalar-LWW half scoped by the red-team — BUG-578: the HLC is unwired, so cross-clone scalar convergence is not deterministic), calibration’s *record + compare* legs (its self-correcting strong form reduced to conjecture by the red-team — the *tune* leg is unbuilt), the incentive-persistent gap, and the new substrate-shapes-output reading — are the probe’s durable output and generalize across the whole family.

Tier 4b — the experimental program, and the method itself (a distinct output). This round produced not just refined claims but a *repeatable apparatus* for testing them, which generalizes beyond AIDA:

- **Five runs.** (1) A prescriptive-brief Claude-vs-Codex bake-off (2026-06-17-competitive-claude-vs-codex.md): Claude won 28/30 to 22/30; the contest measured conscientiousness, not creativity — the loser lost at the fine print, skipping a named gate. (2) An open-brief bake-off (2026-06-18-open-brief-convergence.md): the conjecture that open briefs diverge was falsified — convergence is substrate-driven, and competition’s payoff is quality-variance QA. (3) A gate-vs-rule ablation, **five controlled cells** — an immediate format invariant (2026-06-18-gate-vs-rule-pilot.md), a buried semantic one (2026-06-18-gate-vs-rule-i2.md, n=10/arm), that same buried one re-run with Codex (cross-vendor I2), a controlled *procedural/tool-use* one (2026-06-18-gate-vs-rule-i3.md, n=10/arm), and a *complex* multi-file/multi-function procedural task with the rule buried among ~8 ambient instructions (2026-06-18-gate-vs-rule-i4.md, Claude n=10/arm): rule-only compliance was 100% in *all five*, the gate did zero work in all five, and **five** pre-registered single-variable conjectures fell in sequence — I1 falsified rules-just-fail, I2 falsified attention-distance, cross-vendor I2 falsified vendor, I3 falsified invariant-type, and I4 falsified task-complexity. With every controlled cell at the ceiling across two vendors and trivial→complex, the conclusion is definitive and methodological: **a clean ablation cannot reproduce rule-dropping at all** — not a sixth theory. The lone drop (the bake-off) lives in the unmeasured real-repo/long-autonomy regime a controlled design cannot reach, so the decisive next step is **field instrumentation of real drains, not a sixth ablation**. (4) A roll-your-own friction benchmark (2026-06-19-ryo-friction-benchmark.md, TASK-871, deterministic grading, same specs at 10/50/200/500, two arms, five tasks incl. two grep-favoring fairness tasks): the typed graph pays on relational queries (~80× latency at 500) and rename blast-radius (silent ref-rot prevented at every scale ≥ 50), grep wins on full-text + flat-list — the P2b “dead weight” reading falsified for relational/rename work, conceded for textual, on capability-friction with the operator-discipline confound named not escaped. (5) A cross-vendor portability pilot (2026-06-19-cross-vendor-portability.md, TASK-870, deterministic grading): a cold Codex (gpt-5.5) joined a live fleet with the stock CLI alone and rode discover→claim→ship→verify to a trailered auto-completed spec in ~50 s with 0 lines of integration — moving P8b’s onboarding-cost premise from argument toward evidence (n=1 vendor, n=1 trivial task), and filing a brief-template path bug (BUG-583) it found en route.
- **The method as a contribution in its own right.** A **headless cross-vendor bake-off harness** — one orchestrator drives Claude (claude -p) and Codex (codex exec) in parallel in isolated git worktrees from an identical brief, with a rubric judge — plus a **deterministically-graded ablation runner** (scripts/ablations/gate-vs-rule.sh: throwaway repo, one toggled variable, regex grading, no judge, pre-registered interpretation; CLAUDE_BIN-parameterized so the same runner drives Codex for the cross-vendor cell). The deterministic-grading design is what lets a probe *embarrass its own thesis* — the gate-vs-rule pilot weakened the blanket P1, I2 then falsified our pre-registered attention-distance conjecture, cross-vendor I2 falsified the *next* one (vendor), I3 falsified the *third* (invariant-type), and I4 falsified the *last* one standing (task-complexity); the credibility of all five cells — and of the terminus they force — rests on the apparatus being one that *could* go against us, which on each conjecture it did. Both harnesses are reusable for any invariant or vendor; the asymmetry surfaced en route (Antigravity has no headless CLI, so it can only be the human-in-the-loop arm) is itself an L5 datapoint: *cross-vendor coordination is only as headless as the least-scriptable vendor*.

These are pilots (n=1 task per bake-off run; n=10/arm for each of the five ablation cells; the RYO benchmark and the portability run both deterministically graded — no judge — but single-operator/synthetic specs and n=1 vendor/trivial-task respectively, so each carries its own named confound: the RYO benchmark measures capability-friction not team-adoption, the

portability run is an existence-proof for one vendor pair not a study; self-evaluating judge on the bake-offs; the five ablation cells span trivial→complex and two vendors, so they replicate a genuine finding — a capable model honors a stated rule at the ceiling across every controllable condition — but reach a terminus rather than locating what makes a rule drop; the one uncontrolled drop is a *complex, real-repo* task, leaving **field instrumentation of real drains** — not a sixth ablation — as the outstanding decisive test) — hypotheses to repeat, not settled results — but the *method* is the part that survives regardless of how the numbers move.

Landscape map (June 2026 snapshot; Y / ~ partial / N; star counts point-in-time):

Tool / family	Typed graph	Code↔spec trace	Lifecycle-authority gate	Roles / leases	Git-canonical (plain files)	Cross-vendor	Durable coordination model
AIDA	Y	Y	Y	Y	Y (git-YAML)	Y	Y
Gas Town + Beads	Y	~ (issue IDs)	Y (merge queue)	Y (7 roles)	N (Dolt DB)	Y	Y
GNAP (git-native)	N	N	N	N	Y	Y	~ (primitive sync)
graft (lock-manager)	~ (runtime DAG)	N	~ (resource locks)	~ (file leases)	N (in-mem bus)	~ (Claude-first)	runtime only
Kiro (spec IDE)	N	N	~ (work-flow seq)	N	Y (md)	N	N (single agent)
Kilo Code	N	N	N	N	N (ephemeral)	Y (model-level)	~ (subtask)
Claude Squad / Conductor	N	N	N	N	Y (work-trees)	Y	N (fan-out)
spec-kit / SPEC-TRE	N	N	~ (phase seq)	N	Y	mixed	N

The synthesis (revised). AIDA’s encapsulated value is *not* a unique architecture — the landscape contains a more-mature sibling. It is (a) a narrow set of distinctive choices within a real family — code-to-spec traceability, git-YAML purity, lifecycle-authority gating, the MCP graph surface — and (b) the knowledge the build produced (Tier 4). For the central research question, the load-bearing landscape fact is that **the typed-graph + roles + gates + cross-vendor coordination model is now independently held by at least two efforts (AIDA, Gas Town/Beads) plus thinner ones (GNAP)** — which *strengthens* P8a (the niche is real and being filled by OSS, not by model vendors) while *sharpening* P8b’s third condition: “out-execute the incumbents” must now also mean out-executing Gas Town/Beads, which has Yegge’s distribution and an order of magnitude more traction (traction that, by peer accounts, may outrun its engineering maturity — see the caution above). The probe’s honest verdict on AIDA-the-artifact tilts further toward “a useful instrument that taught us the problem,” not “the thing to bet on” — exactly the distinction the paper exists to keep clear.

A full, dated roster of the surrounding market — ~50 players across frontier agents, IDEs, orchestrators, spec/record tools, SaaS trackers, and standards, each with an honest substance-vs-hype note — is maintained alongside this paper in 2026-06-17-agent-coordination-market-landscape.md.

Appendix A — Proposition ↔ layer ↔ mechanism ↔ grade

P	Layer	Claim (one line)	Mechanism	Grade
P2a	L1	<i>Holds:</i> a stable, mutually-addressable name per work-unit is necessary fleet infrastructure (code symbols are not work-unit handles)	stable IDs across sessions/vendors	(M)
P2b	L1	<i>Relational/rename half now controlled, textual half conceded, team-adoption half still open:</i> RYO friction benchmark (TASK-871) — graph pays on relational queries (~80× at 500) + rename blast-radius (silent ref-rot prevented at every scale ≥ 50), grep wins on full-text + flat-list; “dead weight” falsified for relational/rename, conceded for textual — capability-friction only, operator-discipline confound named not escaped	typed relationships, traces, MCP; RYO friction benchmark	(D)

P	Layer	Claim (one line)	Mechanism	Grade
P1	L2	<i>Terminus (strong negative):</i> across every controlled condition — five cells, two vendors, output-shape + procedural, near + far, trivial→complex — a capable model honors a stated rule at the ceiling and the gate buys nothing (all five conjectures falsified: rules-fail/I1, attention-distance/I2, vendor/cross-vendor I2, invariant-type/I3, task-complexity/I4). A clean ablation can't reproduce rule-dropping at all; the gate's case survives only in the unmeasured real-repo regime → pivot to field instrumentation, not a sixth ablation	gated transitions, merge-gate, trailer-completes; gate-vs-rule pilot + I2 + cross-vendor I2 + I3 + I4 ablations	(M)(D)
P5a	L2	<i>Holds:</i> record + compare are built and useful (predicted-vs-actual gap made visible)	calibration mode + findings calibration --stats	(M)(D)

P	Layer	Claim (one line)	Mechanism	Grade
P5b	L2	<i>Conjecture, not finding:</i> self-correcting beats static — the <i>tune</i> leg is unbuilt; “60%-learner beats static-80%” is illustrative, never measured; ground truth is subjective + estimand non-stationary	(no learning estimator exists)	(M, conjecture)
P4a	L3	<i>Holds:</i> append-union / G-Set by UUID keeps state conflict-light, no clock needed	merge_spec_three_ (M) union of his- tory/comments/processing_record	(D)
P4b	L3	<i>Rebutted as stated:</i> scalar merge is wall-clock LWW, ties → “ours” (conflict.rs:199), non- deterministic across clones; the HLC is built but unwired (zero non-test call sites) — not the cross-node total order the paper claimed	conflict.rs:199; (D) hlc.rs dead (BUG-578)	
P3	L4	<i>Scoped:</i> under stateless escalation, substrate is the cheapest autonomy lever	headless advisor cold-boot + punt routing	(M)(D)
P6	L4	Punt-and- continue role cascade for throughput	EPIC-28 resilient drain, autonomy ladder	(M)(D)
P8a	L5	Vendors are disincentivized to build <i>portable</i> coordination	vendor-neutral mail- box/roles/leases over CLI+MCP	(M)

P	Layer	Claim (one line)	Mechanism	Grade
P8b	L5	<i>Contingent</i> : a durable layer is defensible by AIDA (rests on 3 named bets); the <i>onboarding-cost</i> premise underneath them now has pilot evidence — a cold Codex joined with 0 lines of integration, shipped a trailered auto-completed spec in ~50 s (TASK-870, n=1 vendor/trivial task); the incumbent-capture hole is untouched by it	same, + the §9 conditions; cross-vendor portability pilot	(D)(K)
P7	surface	Defer depth-disclosure to use	TUI-as-product	(D)(K)

Working draft. To revise with: collaborators' challenges to the propositions, real numbers from any of the §13 ablations, and updated market snapshots (dated, immutable).